

需求分析与功能设计

目标用户：读者（浏览，借还，查看个人记录）、管理员（管理图书与库存，查看借阅统计）、系统（到期提醒，报表）

困难：模糊查询没匹配，借阅时间超过月份

功能列表：用户注册&登录、浏览图书列表、借书/还书、个人借阅记录、管理员编辑图书、基础数据库与迁移。

优先要做：图书副本管理（处理并发）、搜索、缓存（redis）、单元/集成/并发测试

数据库设计

用户表：

id是bigint数据类型，而且auto_increment自动增长 且是主键

username是唯一的 可变字长 不可空

角色默认是普通用户 必填

```
-- users
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(100) NOT NULL UNIQUE,
  email VARCHAR(255) DEFAULT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(32) NOT NULL DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

图书表：

id,isbn,title,author,summary,publisher,publish_date

total_copies:书籍数量 available_copies 可借阅的数量

记录书籍创建时间 更新时间

`FULLTEXT KEY ft_title_author_summary (title, author, summary)`:全文索引，包含title, author, summary。用于对这三个字段的模糊搜索，比like更高效。（!!! 要解释自己的优化方案）

对isbn和title做索引

```
-- books (主表)
CREATE TABLE books (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  isbn VARCHAR(32) DEFAULT NULL,
  title VARCHAR(255) NOT NULL,
  author VARCHAR(255) DEFAULT NULL,
  summary TEXT,
  publisher VARCHAR(255) DEFAULT NULL,
  publish_date DATE,
  total_copies INT NOT NULL DEFAULT 0,
  available_copies INT NOT NULL DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  FULLTEXT KEY ft_title_author_summary (title, author, summary)
) ENGINE=InnoDB;

CREATE INDEX idx_books_isbn ON books(isbn);
CREATE INDEX idx_books_title ON books(title);
```

图书副本表:

每个副本自己的id, 关联图书主表的id

```
-- book_copies (每个副本一行)
CREATE TABLE book_copies (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  book_id BIGINT NOT NULL,
  barcode VARCHAR(64) DEFAULT NULL, -- 副本的条形码或编号, 用于扫描借还--
  copy_status TINYINT NOT NULL DEFAULT 1, -- 副本当前状态 1 avail, 2
borrowed, 3 reserved
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- 副本创建时间--
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP, -- 副本最后更新时间--

  -- 外键约束: book_id关联books表的id--
  CONSTRAINT fk_copy_book FOREIGN KEY (book_id) REFERENCES
books(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 为常用查询建复合索引 (顺序重要) 用于查询某本书的所有可借阅副本
CREATE INDEX idx_copy_book_status ON book_copies (book_id,
copy_status);
```

```
-- 若常按照 status, book_id 筛选, 可考虑单列索引
```

借阅图书副本的信息表:

`borrow_records` 的表, 用于记录用户借阅图书副本的详细信息 (如借阅时间、应还时间、实际归还时间等), 并定义了外键约束和索引.

`borrow_records` 表是连接用户、图书副本、图书主表的核心表, 完整记录了借阅行为的关键信息 (时间、状态等)。外键约束通过不同策略 (级联删除 / 限制删除) 保证数据一致性, 索引则针对高频查询场景 (如用户借阅历史、副本流转记录) 优化性能, 适合图书馆等需要严格管理借阅流程的系统。

```
-- borrow_records
CREATE TABLE borrow_records (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL, --关联的用户表 users 的 id
  copy_id BIGINT NOT NULL, --关联的图书副本表 book_copies 的 id
  book_id BIGINT NOT NULL, --关联的图书主表 books 的 id
  borrow_time DATETIME NOT NULL, --借阅发生的时间
  due_time DATETIME NOT NULL, --应归还时间
  return_time DATETIME NULL, --实际归还时间
  status TINYINT NOT NULL DEFAULT 1, -- 借阅记录的状态 1 borrowed, 2
returned, 3 overdue
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, --记录创建时间
  --外键 fk_br_user 关联 users 表的 id, 表示借阅记录属于某用户。
  CONSTRAINT fk_br_user FOREIGN KEY (user_id) REFERENCES
users(id) ON DELETE CASCADE,
  --外键 fk_br_copy 关联 book_copies 表的 id, 表示记录对应某副本
  CONSTRAINT fk_br_copy FOREIGN KEY (copy_id) REFERENCES
book_copies(id) ON DELETE RESTRICT
) ENGINE=InnoDB;
--复合索引, 包含 user_id 和 status, 优化高频查询如 “查询某用户
(user_id=10) 的所有未归还借阅 (status=1)
CREATE INDEX idx_br_user_status ON borrow_records (user_id,
status);
--单列索引, 优化 “查询某副本 (copy_id=5) 的所有借阅历史”
CREATE INDEX idx_br_copy ON borrow_records (copy_id);
--单列索引, 优化 “查询某本书 (book_id=3) 的所有借阅记录”, 利用冗余字段
book_id 快速筛选, 无需关联 book_copies 表。
CREATE INDEX idx_br_book ON borrow_records (book_id);
```

预约表:

`reservations` 的表, 用于记录用户对图书的预约信息 (当图书副本全部被借出时, 用户可预约等待)

```
-- reservations
CREATE TABLE reservations (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,--关联的用户表 users 的 id
  book_id BIGINT NOT NULL,--关联的图书主表 books 的 id
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,--预约记录的创建时间
  status VARCHAR(32) DEFAULT 'pending', -- 预约的状态pending待处理|notified已通知|cancelled已取消
  --外键约束
  CONSTRAINT fk_res_user FOREIGN KEY (user_id) REFERENCES
  users(id) ON DELETE CASCADE,
  CONSTRAINT fk_res_book FOREIGN KEY (book_id) REFERENCES
  books(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

系统信息表：

`messages` 的表，用于存储系统向用户发送的站内消息（如预约通知、逾期提醒

`channel VARCHAR(32) DEFAULT 'in_app':channel`：消息发送渠道，`DEFAULT 'in_app'`：默认值为 `in_app`（站内消息）也可以选择短信邮件

```
-- messages (in-app)
CREATE TABLE messages (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,--关联的用户表 users 的 id
  title VARCHAR(255),--消息标题
  content TEXT,--消息的详细内容
  channel VARCHAR(32) DEFAULT 'in_app',
  status VARCHAR(32) DEFAULT 'pending',--消息状态
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  sent_at TIMESTAMP NULL,--消息实际发送成功的时间
  CONSTRAINT fk_msg_user FOREIGN KEY (user_id) REFERENCES
  users(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

索引设计（汇总）

聚簇索引（主键）、唯一索引（值唯一）

复合索引

- `(book_id,copy_status)`：借书的时候，查询 `WHERE book_id = ? AND copy_status = 1 LIMIT 1` 定位到可借的副本
- `(user_id,satus)`：查询某用户借阅历史。个人查看自己借阅记录

覆盖索引：覆盖索引指索引包含了查询需要的所有列，MySQL 可以只读索引无需回表，从而减少 IO，速度快。典型做法是将常查询的少量列放入复合索引中实现覆盖。

全文索引 (FULLTEXT, 查询 MATCH)

- `FULLTEXT(ft_title_author_summary)`：用于全文搜索 (MySQL FULLTEXT)。注意：中文分词不佳，长期建议 ES。
- `idx_copy_book_status (book_id, copy_status)`：借书时常做 `WHERE book_id = ? AND copy_status = 1 LIMIT 1`，这个复合索引可覆盖查询并使用索引顺序。
- `idx_br_user_status (user_id, status)`：用户查看未归还、历史记录时按 `user_id + status` 查。
- `idx_books_isbn`：ISBN 精确查找。
- 覆盖索引 (Covering Index)：在一些高频读场景，选择少量列创建复合索引以避免回表 (例如查询列表只需 id,title,author, 可以用索引扫描)。
- 索引顺序原则：最常被 filter 的列放在最前；用于排序的列也要考虑 (如果 ORDER BY 可以走索引)。

并发/锁与事务 (借书场景详解)

目标：在并发环境下，确保同一副本不会被两个用户同时借走；防止超借

1) 悲观锁，事务，行锁：行加行级排他锁，防止其它事务同时拿到同一副本。

```
START TRANSACTION;
```

```
-- 1) 找到一个可用副本并锁住该行 (行级锁)
```

```
SELECT id FROM book_copies
WHERE book_id = ? AND copy_status = 1
ORDER BY id ASC
LIMIT 1
FOR UPDATE;
```

```
-- 若没有返回行 -> 没有可借副本，ROLLBACK并返回失败
```

```
-- 2) 标记副本为 borrowed
```

```
UPDATE book_copies SET copy_status = 2 WHERE id = <copy_id>;
```

```
-- 3) 插入 borrow_records
```

```
INSERT INTO borrow_records(user_id, copy_id, book_id,
borrow_time, due_time, status)
VALUES(...);
```

```
-- 4) 更新 books.available_copies = available_copies - 1
```

```
UPDATE books SET available_copies = available_copies - 1 WHERE id = ?;  
  
COMMIT;
```

2) 乐观锁方案 (可选, 高并发下减少锁开销)

- 给 `book_copies` 或 `books` 增 `version` 列 (INT) 。借书时: 读取版本 `v`;
UPDATE ... SET available_copies = available_copies -1, version = version+1
WHERE id = ? AND version = v; 如果受影响行数 = 0, 说明冲突, 重试/告知失败。
- 优点: 不加持久锁, 适合冲突低场景; 缺点: 需处理重试和补偿。

MyBatis

mybatis在图书管理项目中是负责把SQL映射到java对象。我们用它来写可控、可优化的sql。比JDBC少得多的样例代码。

在结构上 controller -- service -- mapper (mybatis) --db

把sql (写在xml或注解) 映射成java方法, 返回实体

1) Mapper 接口 + XML (核心)

```
@Mapper  
public interface BookCopyMapper {  
    Long selectAvailableCopyForUpdate(@Param("bookId") Long  
bookId);  
    int updateCopyStatus(@Param("id") Long id, @Param("status") int  
status);  
}
```

XML (`mapper/BookCopyMapper.xml`) :

```
<select id="selectAvailableCopyForUpdate" resultType="long"  
parameterType="long">  
    SELECT id FROM book_copies  
    WHERE book_id = #{bookId} AND copy_status = 1  
    ORDER BY id ASC  
    LIMIT 1  
    FOR UPDATE  
</select>
```

说明： `FOR UPDATE` 在事务内会加行级锁，保证并发安全；这是 MyBatis 把 SQL 能力直接暴露出来的好处。

2) Service 层使用 Mapper（事务注解）

```
@Service
public class BorrowServiceImpl implements BorrowService {
    @Resource
    private BookCopyMapper bookCopyMapper;

    @Transactional
    public Long borrowBook(Long userId, Long bookId) {
        Long copyId =
            bookCopyMapper.selectAvailableCopyForUpdate(bookId);
        if (copyId == null) throw new RuntimeException("no available
            copy");
        bookCopyMapper.updateCopyStatus(copyId, 2);
        // insert record, update books.availableCopies...
        return recordId;
    }
}
```

说明： Mapper 专注 SQL，Service 管理事务与业务流程（组合多个 mapper 操作），职责清晰。

把请求链路分层说明，面试说清楚每层职责与 MyBatis 的位置：

```
Controller（接收 HTTP 请求）
    → Service（业务逻辑、事务边界 @Transactional）
        → MyBatis Mapper（SQL 层，XML/注解写 SQL）
            → JDBC（由 MyBatis 管理 Connection/PreparedStatement）
                → MySQL（数据库）
```

同时补充其他模块如何参与：

- **Config：** `DataSource`（HikariCP）、`@MapperScan`、MyBatis 配置（`typeAliases`、`mapperLocations`）。
- **Flyway：** 做 schema migration（DB 表结构由 Flyway 管理，与 MyBatis 配合）。
- **事务管理：** 由 Spring 的 `DataSourceTransactionManager` 管理，`@Transactional` 在 Service 层控制事务边界。
- **测试：** Testcontainers 启 DB，SpringBootTest + `@MapperScan` 自动装配 Mapper，执行集成测试。

一句话总结：MyBatis 是连接 Service 与 MySQL 的桥梁，负责执行 SQL 与把结果映射回 Java，对外只暴露接口方法给 Service 使用。

1. Q: MyBatis 是如何防止 SQL 注入的？

A (示例答)：使用 `#{} (PreparedStatement 占位)` 会自动用参数绑定从而防止注入；`${}` 是字符串拼接，不安全，只有在拼接列名/排序字段时谨慎使用并做白名单校验。

2. Q: 你们怎么做分页？

A：我们用 PageHelper 或自己用 `LIMIT #{offset}, #{limit}`。对于大页号改用 Keyset pagination (`WHERE id > lastId LIMIT n`) 以避免 offset 性能问题。

3. Q: Mapper 的 XML 与注解哪个更好？

A：XML 适合复杂 SQL 和维护（分离层次）；注解适合简单 SQL。我们用 XML 来管理复杂的查询（比如 search、FOR UPDATE、JOIN），注解用于很短的 CRUD。

4. Q: 如何做动态 SQL？

A：使用 MyBatis 的 `<if>`, `<choose>`, `<foreach>` 在 XML 中生成条件和 IN 列表，方便又清晰。例如搜索筛选、批量 insert/更新会用 `<foreach>`。

5. Q: MyBatis 的一级/二级缓存是怎样的？

A：一级缓存基于 SqlSession（同一次 session 内可复用），默认开启；二级缓存是 mapper 级别的（需显式开启并配置缓存实现），适用于读多写少场景。生产中要慎用二级缓存，注意事务一致性与缓存失效策略。

6. Q: 如何处理大事务或死锁？

A：在 Service 层控制事务粒度，保持事务尽可能短；捕获死锁异常（如 `ER_LOCK_DEADLOCK`），做指数退避重试；优化 SQL 和索引以减少锁争用。

7. Q: 为什么不直接用 JPA/Hibernate？

A (简短)：JPA 自动化方便，但对复杂查询/性能调优不够透明。我们需要精确控制 SQL（`FOR UPDATE`、复杂聚合、手写优化），所以选 MyBatis。

8. Q: 如何单元测试 Mapper？

A：Mapper 可以在 SpringBootTest 环境下与 Testcontainers 的 MySQL 做集成测试；单元层可以 mock Mapper (Mockito) 测试 Service 逻辑。

9. Q: Mapper XML 的 resultMap 有什么用？

A：当列名和属性名不一致或有复杂关联 (one-to-many、nested result) 时，用 resultMap 指定精确映射，避免类型/列名映射错误。

10. Q: 如何处理 ENUM/特殊类型映射？

A：用 MyBatis 的 TypeHandler (自定义) 处理枚举与 DB 字段转换，或用 MyBatis 的 `typeHandlersPackage` 全局配置。

• “在高并发下 MyBatis 与 Redis 结合的模式是什么？”

答：常见是 Redis 预减库存 (atomic DECR)，若成功再请求 DB 写入并用消息

队列最终一致性补偿；复杂逻辑用 SQL 行锁（悲观锁）在 DB 层做强一致，Redis 做读缓存/热点保护。可根据并发量和一致性需求权衡。

- “如何在 MyBatis 中做批量插入/更新？”

答：用 `<foreach>` 生成批量 `INSERT`，并设置 `useGeneratedKeys` 或返回主键。也可借助 MyBatis-Plus 或 MyBatis Generator。

- “如何监控 SQL 执行？”

答：在项目中启用慢查询日志、使用 p6spy 或 datasource proxy 打印 SQL（带参数），并结合 APM（SkyWalking）和 Prometheus/Grafana 监控 DB 指标。

“选 MyBatis 是因为我们既要**掌控 SQL 性能与并发行为**（比如借书的 `FOR UPDATE`），又要**减少原生 JDBC 的样板代码**，在项目里它通过 Mapper 把 SQL 映射为方法，Service 层注入这些 Mapper 并在 `@Transactional` 边界内组合调用，形成 Controller→Service→Mapper→DB 的清晰链路，这样既满足可维护性又满足性能调优能力。”

详细流程

好 — 既然你的 **Mapper 已经做好了**（假设有 `search(String q, int offset, int limit)` 和 `countSearch(String q)` 两个方法），下面我给你**几种可直接拷进项目的 Service 层实现**，每种都包含：

- 接口（`BookService`）
- 返回类型（`PageResult<T>`，便于前端使用）
- 两个实现：
 1. **手写 LIMIT+COUNT（通用且明确）**
 2. **PageHelper 插件版本（更少代码，企业常用）**
- Controller 调用示例
- 额外：**Keyset（cursor）分页**的实现思路与示例方法（高并发/大数据量用）

你可以任选一种接入到项目里。代码都以你项目的 package（`org.example.library`）为例。

1) 通用分页返回 DTO (放

src/main/java/org/example/library/dto/PageResult.java)

```
package org.example.library.dto;

import lombok.AllArgsConstructor;
import lombok.Data;
import java.util.List;

@Data
@AllArgsConstructor
public class PageResult<T> {
    private List<T> list;
    private long total;    // 总条数
    private int page;      // 当前页 (1-based)
    private int size;      // 每页大小
}
```

2) BookService 接口 (放 service/BookService.java)

```
package org.example.library.service;

import org.example.library.dto.PageResult;
import org.example.library.entity.Book;

public interface BookService {
    PageResult<Book> searchBooks(String q, int page, int size);
    // keyset 分页接口 (可选)
    PageResult<Book> searchBooksByCursor(String q, Long lastId,
    int size);
}
```

3A) 实现一 — 手写 LIMIT + COUNT (放

service/impl/BookServiceImpl.java)

适合你已经有 mapper 的场景，最明确、最可控。

```
package org.example.library.service.impl;

import org.example.library.dto.PageResult;
import org.example.library.entity.Book;
import org.example.library.mapper.BookMapper;
import org.example.library.service.BookService;
```

```

import org.springframework.stereotype.Service;
import jakarta.annotation.Resource;

import java.util.Collections;
import java.util.List;

@Service
public class BookServiceImpl implements BookService {

    @Resource
    private BookMapper bookMapper;

    @Override
    public PageResult<Book> searchBooks(String q, int page, int
size) {
        if (page < 1) page = 1;
        if (size <= 0) size = 10;
        int offset = (page - 1) * size;

        // 防止 null 导致 mapper SQL 问题
        String keyword = (q == null) ? "" : q.trim();

        List<Book> list = Collections.emptyList();
        long total = 0;
        // 当关键字为空且不想走 count 或特殊处理时可以优化，这里直接统一走两
条查询
        list = bookMapper.search(keyword, offset, size);
        total = bookMapper.countSearch(keyword);

        return new PageResult<>(list, total, page, size);
    }

    @Override
    public PageResult<Book> searchBooksByCursor(String q, Long
lastId, int size) {
        // 见下面 keyset 实现示例
        if (size <= 0) size = 10;
        String keyword = (q == null) ? "" : q.trim();

        List<Book> list;
        if (lastId == null || lastId <= 0) {
            // 首页: 无 lastId, 直接取最新的 size 条
            list = bookMapper.searchByKeysetFirstPage(keyword,
size);
        } else {

```

```

        list = bookMapper.searchByKeysetAfterId(keyword,
lastId, size);
    }
    // Keyset 不返回总数, 设 total = -1 或返回估算
    return new PageResult<>(list, -1, 0, size);
}
}

```

说明:

- `bookMapper.search(keyword, offset, size)` 和 `countSearch` 为你现有方法。
- `searchBooksByCursor` 调用的是你要额外实现的 Keyset mapper (示例在下文)。
- 这个实现把分页逻辑放在 Service, 便于单元测试与复用。

3B) 实现二 — 使用 PageHelper (放

`service/impl/BookServicePageHelperImpl.java`)

如果你引入 PageHelper 插件, Service 更简洁。先在 `pom.xml` 加依赖 (我之前给过), 然后:

```

package org.example.library.service.impl;

import com.github.pagehelper.PageHelper;
import com.github.pagehelper.PageInfo;
import org.example.library.dto.PageResult;
import org.example.library.entity.Book;
import org.example.library.mapper.BookMapper;
import org.example.library.service.BookService;
import org.springframework.stereotype.Service;
import jakarta.annotation.Resource;

import java.util.List;

@Service
public class BookServicePageHelperImpl implements BookService {

    @Resource
    private BookMapper bookMapper;

    @Override
    public PageResult<Book> searchBooks(String q, int page, int
size) {

```

```

        if (page < 1) page = 1;
        if (size <= 0) size = 10;
        String keyword = (q == null) ? "" : q.trim();

        PageHelper.startPage(page, size);
        // 这里 mapper.search 参数可调整: 若 mapper 定义了
        offset/limit 参数, 可忽略或传 0
        List<Book> list = bookMapper.search(keyword, 0, 0);
        PageInfo<Book> pageInfo = new PageInfo<>(list);

        return new PageResult<>(pageInfo.getList(),
        pageInfo.getTotal(), pageInfo.getPageNum(),
        pageInfo.getPageSize());
    }

    @Override
    public PageResult<Book> searchBooksByCursor(String q, Long
    lastId, int size) {
        // 同上 keyset 处理
        if (size <= 0) size = 10;
        String keyword = (q == null) ? "" : q.trim();
        List<Book> list;
        if (lastId == null || lastId <= 0) {
            list = bookMapper.searchByKeysetFirstPage(keyword,
            size);
        } else {
            list = bookMapper.searchByKeysetAfterId(keyword,
            lastId, size);
        }
        return new PageResult<>(list, -1, 0, size);
    }
}

```

说明: PageHelper 会拦截 Mapper 的查询并自动注入 `LIMIT`, 并自动执行 `COUNT` 获取 total (默认行为)。PageHelper 常见于企业项目, 省事省代码。

4) Controller 调用示例 (放 `controller/BookController.java`)

```

package org.example.library.controller;

import org.example.library.common.Result;
import org.example.library.dto.PageResult;
import org.example.library.entity.Book;
import org.example.library.service.BookService;

```

```

import org.springframework.web.bind.annotation.*;
import jakarta.annotation.Resource;

@RestController
@RequestMapping("/api/books")
public class BookController {

    @Resource
    private BookService bookService;

    // 普通分页
    @GetMapping
    public Result<?> list(@RequestParam(defaultValue = "") String
q,
                        @RequestParam(defaultValue = "1") int
page,
                        @RequestParam(defaultValue = "10") int
size) {
        PageResult<Book> result = bookService.searchBooks(q,
page, size);
        return Result.success(result);
    }

    // cursor 分页 (infinite-scroll 适用)
    @GetMapping("/cursor")
    public Result<?> cursorList(@RequestParam(defaultValue = "")
String q,
                                @RequestParam(required = false)
Long lastId,
                                @RequestParam(defaultValue =
"10") int size) {
        PageResult<Book> result =
bookService.searchBooksByCursor(q, lastId, size);
        return Result.success(result);
    }
}

```

5) Keyset (游标) 分页: Mapper 示例 SQL (把这两条加到你的 BookMapper.xml)

```

<!-- 首页: 取最新的 N 条 -->
<select id="searchByKeysetFirstPage" resultMap="BookMap">
    SELECT id, isbn, title, author, available_copies
    FROM books

```

```
WHERE (title LIKE CONCAT('%', #{q}, '%') OR author LIKE
CONCAT('%', #{q}, '%') OR isbn LIKE CONCAT('%', #{q}, '%'))
ORDER BY id DESC
LIMIT #{size}
</select>
```

<!-- 翻页: 取 id < lastId 的下一页 -->

```
<select id="searchByKeysetAfterId" resultMap="BookMap">
  SELECT id, isbn, title, author, available_copies
  FROM books
  WHERE (title LIKE CONCAT('%', #{q}, '%') OR author LIKE
CONCAT('%', #{q}, '%') OR isbn LIKE CONCAT('%', #{q}, '%'))
    AND id < #{lastId}
  ORDER BY id DESC
  LIMIT #{size}
</select>
```

注意: Keyset 在 `ORDER BY id DESC` 且 `id` 为自增主键时最稳定; 若你按其他列排序, Keyset 要用复合条件并确保排序列上有索引。

6) 单元测试建议 (容易在面试演示)

- 单元测试 `BookServiceImpl`: mock `BookMapper`, 验证当 `page=1, size=10` 时 `offset=0` 被传入 mapper 的 `search`, 以及当 mapper 返回 list 和 count 时正确构造 `PageResult`。用 Mockito 做断言。
- 集成测试: 用 Testcontainers 启动 MySQL, seed 若干数据, 调用 Controller 接口或 Service 方法, 验证分页正确性、总数一致性、边界 (最后一页) 行为。

7) 面试时你可以这样说 (简短模板)

- “我们在 MVP 里采用 `LIMIT offset, size + COUNT(*)` 的方式实现分页, Service 层负责计算 `offset=(page-1)*size` 并调用 Mapper 的 `search` 与 `countSearch`。这是实现简单、逻辑清晰的方案。为应对大偏移性能问题, 我们准备了 Keyset 游标分页 (`id < lastId LIMIT n`) 的实现, 并在未来会基于使用场景切换到该方案或引入 Elasticsearch 做搜索分页优化。”
- 如果被问到“PageHelper”: 说“生产中常用 PageHelper 来简化代码, 它自动插入 LIMIT 并封装 total; 我们项目也支持该方案, 只要在 `pom.xml` 加依赖并在 Service 里 `PageHelper.startPage(page, size)` 即可”。

Redis缓存

Redis 用来缓解数据库读压力、加速热点数据访问（比如图书详情页、热门榜），并能做简单的计数/排名（ZSET）来实现热度排行榜——对比直接读 MySQL，Redis 的响应更快（内存）、并支持原子操作。

下面用 `RedisTemplate<String, String>` + Jackson 将对象序列化为 JSON 存储。也展示用 Redis 的 ZSET 维护热度。

1) RedisConfig.java

```
package org.example.library.config;
// 导入Jackson的ObjectMapper，用于JSON序列化和反序列化
import com.fasterxml.jackson.databind.ObjectMapper;
// 导入Jackson的序列化特性类，用于配置日期序列化方式
import com.fasterxml.jackson.databind.SerializationFeature;
// 导入Spring的Bean注解，用于定义Bean
import org.springframework.context.annotation.Bean;
// 导入Spring的Configuration注解，标识当前类为配置类
import org.springframework.context.annotation.Configuration;
// 导入Redis连接相关的类
import org.springframework.data.redis.connection.*;
// 导入Lettuce连接工厂，用于创建Redis连接
import
org.springframework.data.redis.connection.lettuce.LettuceConnecti
onFactory;
// 导入Redis核心操作类
import org.springframework.data.redis.core.*;
// 导入Redis序列化器相关类
import org.springframework.data.redis.serializer.*;

// 标识当前类是Spring的配置类，会被Spring扫描并加载其中的Bean定义
@Configuration
public class RedisConfig {

    // 定义Redis连接工厂的Bean，用于创建Redis连接
    @Bean
    public LettuceConnectionFactory redisConnectionFactory() {
        // 注释说明：生产环境应该通过配置文件来配置Redis的主机、端口和密码，
        这里为了示例直接硬编码
        // 创建Redis单机配置对象，设置Redis服务器地址为localhost，端口为
        6379
        RedisStandaloneConfiguration cfg = new
        RedisStandaloneConfiguration("localhost", 6379);
        // 根据配置创建并返回Lettuce连接工厂
        return new LettuceConnectionFactory(cfg);
    }
}
```


// 定义RedisTemplate的Bean，用于操作Redis，依赖注入Lettuce连接工厂和
ObjectMapper

@Bean

```
public RedisTemplate<String, String>  
redisTemplate(LettuceConnectionFactory connectionFactory,  
ObjectMapper mapper) {  
    // 创建RedisTemplate实例，泛型指定键和值都是String类型  
    RedisTemplate<String, String> rt = new RedisTemplate<>();  
    // 设置Redis连接工厂，指定该模板使用的连接工厂  
    rt.setConnectionFactory(connectionFactory);  
  
    // 创建字符串序列化器，用于序列化Redis的键和值  
    StringRedisSerializer keySer = new  
StringRedisSerializer();  
    StringRedisSerializer valueSer = new  
StringRedisSerializer();  
  
    // 设置Redis键的序列化器  
    rt.setKeySerializer(keySer);  
    // 设置Redis值的序列化器  
    rt.setValueSerializer(valueSer);  
    // 设置Redis哈希结构中键的序列化器  
    rt.setHashKeySerializer(keySer);  
    // 设置Redis哈希结构中值的序列化器  
    rt.setHashValueSerializer(valueSer);  
  
    // 初始化RedisTemplate的属性，必须调用此方法完成初始化  
    rt.afterPropertiesSet();  
    // 返回配置好的RedisTemplate实例  
    return rt;  
}
```

// 定义ObjectMapper的Bean，用于JSON处理

@Bean

```
public ObjectMapper objectMapper() {  
    // 创建ObjectMapper实例，Jackson库的核心类，用于JSON序列化和反序  
列化  
    ObjectMapper m = new ObjectMapper();  
    // 查找并注册所有可用的模块（如Java 8日期时间模块），增强JSON处理能  
力  
    m.findAndRegisterModules();  
    // 禁用将日期序列化为时间戳的特性，改为使用ISO格式的字符串（如"2024-  
10-22T12:00:00"）
```

```

        m.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        // 返回配置好的ObjectMapper实例
        return m;
    }
}

```

2) BookCacheService.java (封装缓存逻辑)

路径:

src/main/java/org/example/library/service/BookCacheService.java

```

package org.example.library.service;

import org.example.library.entity.Book;

import java.util.List;

/**
 * 图书缓存服务接口，定义了与图书相关的缓存操作方法
 * 主要用于管理图书数据在Redis等缓存中的存储、获取、更新和删除，提升查询性能
 */
public interface BookCacheService {

    /**
     * 从缓存中获取图书信息
     * @param bookId 图书ID
     * @return 缓存中的图书对象，如果缓存不存在则返回null
     */
    Book getBookFromCache(Long bookId);

    /**
     * 使用缓存旁路模式（Cache-Aside Pattern）获取图书信息
     * 缓存旁路模式逻辑：先查缓存，缓存命中则返回；未命中则查数据库，再将结果写入缓存
     * @param bookId 图书ID
     * @return 图书对象（可能来自缓存或数据库）
     */
    Book getBookwithCacheAside(Long bookId); // cache-aside: 缓存旁路模式注解说明

    /**
     * 清除指定图书的缓存
     * 通常在图书数据更新后调用，避免缓存与数据库数据不一致
     */
}

```

```

    * @param bookId 图书ID
    */
    void evictBookCache(Long bookId);

    /**
     * 将图书信息存入缓存，并设置过期时间
     * @param bookId 图书ID（作为缓存键）
     * @param book 图书对象（作为缓存值）
     * @param ttlSeconds 缓存过期时间（单位：秒），过期后自动删除
     */
    void putBookCache(Long bookId, Book book, long ttlSeconds);

    /**
     * 获取热门图书列表
     * @param topN 要获取的热门图书数量（前N名）
     * @return 热门图书列表，按热度排序
     */
    List<Book> getHotBooks(int topN);

    /**
     * 增加图书的热度评分
     * 用于记录图书的受欢迎程度，为热门图书列表提供数据支持
     * @param bookId 图书ID
     * @param delta 增加的评分值（可以是正数或负数）
     */
    void incrementHotScore(Long bookId, double delta);
}

```

实现类 `BookCacheServiceImpl.java`:

```

```java
package org.example.library.service.impl;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.annotation.Resource;
import org.example.library.entity.Book;
import org.example.library.mapper.BookMapper;
import org.example.library.service.BookCacheService;
import org.springframework.data.redis.core.*;
import org.springframework.stereotype.Service;

import java.time.Duration;
import java.util.*;
import java.util.stream.Collectors;

```

```

// 标记为Spring服务类，用于实现图书缓存相关业务逻辑
@Service
public class BookCacheServiceImpl implements BookCacheService {

 // 缓存键前缀：图书详情缓存的键前缀，格式为"book:detail:图书ID"
 private static final String KEY_BOOK = "book:detail: ";
 // 缓存键：热门图书排序集合的键，用于存储图书热度评分
 private static final String KEY_HOT = "book:hot ";
 // 缓存键前缀：空值占位符的键前缀，用于防止缓存穿透
 private static final String KEY_PLACEHOLDER =
"placeholder:book: ";

 // 注入RedisTemplate，用于操作Redis缓存
 @Resource
 private RedisTemplate<String, String> redisTemplate;
 // 注入ObjectMapper，用于JSON序列化和反序列化
 @Resource
 private ObjectMapper objectMapper;
 // 注入BookMapper，用于从数据库查询图书信息（缓存旁路模式中使用）
 @Resource
 private BookMapper bookMapper; // 直接访问mapper用于cache-aside
模式

 /**
 * 从缓存中获取图书信息
 * @param bookId 图书ID
 * @return 缓存中的图书对象，若缓存不存在或为占位符则返回null
 */
 @Override
 public Book getBookFromCache(Long bookId) {
 // 构建缓存键：前缀 + 图书ID
 String key = KEY_BOOK + bookId;
 // 从Redis中获取对应的值（JSON字符串）
 String json = redisTemplate.opsForValue().get(key);
 // 缓存不存在，直接返回null
 if (json == null) return null;
 // 处理空值占位符：如果缓存值是__NULL__，表示数据库中也不存在该图书，
 返回null
 if ("__NULL__".equals(json)) return null;
 // 将JSON字符串反序列化为Book对象并返回
 try {
 return objectMapper.readValue(json, Book.class);
 } catch (Exception e) {
 // 反序列化失败时抛出运行时异常

```

```

 throw new RuntimeException(e);
 }
}

/**
 * 使用缓存旁路模式（Cache-Aside）获取图书信息
 * 逻辑：先查缓存，命中则返回；未命中则查数据库，再将结果写入缓存
 * @param bookId 图书ID
 * @return 图书对象（来自缓存或数据库）
 */
@Override
public Book getBookWithCacheAside(Long bookId) {
 // 构建缓存键
 String key = KEY_BOOK + bookId;
 // 从缓存获取数据
 String cached = redisTemplate.opsForValue().get(key);
 // 缓存命中
 if (cached != null) {
 // 若为占位符，返回null
 if ("__NULL__".equals(cached)) return null;
 // 反序列化为Book对象并返回
 try { return objectMapper.readValue(cached,
Book.class); }
 catch (Exception e) { throw new RuntimeException(e);
 }

 // 缓存未命中，从数据库查询
 Book b = bookMapper.findById(bookId);
 // 数据库中也不存在该图书
 if (b == null) {
 // 设置空值占位符到缓存（短期过期），防止缓存穿透（大量无效请求直
击数据库）
 redisTemplate.opsForValue().set(KEY_PLACEHOLDER +
bookId, "__NULL__", Duration.ofSeconds(60));
 // 同时在图书详情缓存键下也设置占位符，保持一致性
 redisTemplate.opsForValue().set(key, "__NULL__",
Duration.ofSeconds(60));
 return null;
 }

 // 数据库查询成功，将结果写入缓存
 try {
 // 将Book对象序列化为JSON字符串
 String json = objectMapper.writeValueAsString(b);

```

```

 // 设置缓存过期时间（10~15分钟随机），避免缓存雪崩（大量缓存同时
 过期）

 long ttl = 60 * 10 + new Random().nextInt(60 * 5); //
 10分钟基础上增加0~5分钟随机值

 redisTemplate.opsForValue().set(key, json,
 Duration.ofSeconds(ttl));
 } catch (Exception e) {
 // 序列化或写入缓存失败时可在此处记录日志，不抛出异常避免影响主流
 程
 }
 // 返回数据库查询到的图书对象
 return b;
}

/**
 * 清除指定图书的缓存
 * 通常在图书数据更新后调用，保证缓存与数据库一致性
 * @param bookId 图书ID
 */
@Override
public void evictBookCache(Long bookId) {
 // 删除图书详情缓存
 redisTemplate.delete(KEY_BOOK + bookId);
 // 删除对应的空值占位符缓存
 redisTemplate.delete(KEY_PLACEHOLDER + bookId);
}

/**
 * 将图书信息存入缓存，并指定过期时间
 * @param bookId 图书ID
 * @param book 图书对象
 * @param ttlSeconds 过期时间（秒）
 */
@Override
public void putBookCache(Long bookId, Book book, long
ttlSeconds) {
 try {
 // 将图书对象序列化为JSON字符串
 String json = objectMapper.writeValueAsString(book);
 // 写入缓存并设置过期时间
 redisTemplate.opsForValue().set(KEY_BOOK + bookId,
 json, Duration.ofSeconds(ttlSeconds));
 } catch (Exception e) {
 // 异常时记录日志，不影响主流程
 }
}

```

```

}

/**
 * 获取热门图书列表（按热度排序的前N名）
 * @param topN 要获取的数量
 * @return 热门图书列表
 */
@Override
public List<Book> getHotBooks(int topN) {
 // 获取Redis的ZSet操作对象（用于操作有序集合）
 ZSetOperations<String, String> z =
redisTemplate.opsForZSet();
 // 从热门图书有序集合中获取分数从高到低的前N个图书ID（reverseRange
表示倒序）
 Set<String> ids = z.reverseRange(KEY_HOT, 0, topN - 1);
 // 若没有数据，返回空列表
 if (ids == null || ids.isEmpty()) return
Collections.emptyList();

 // 批量构建图书详情缓存的键（前缀+图书ID）
 List<String> keys = ids.stream().map(id -> KEY_BOOK +
id).collect(Collectors.toList());
 // 批量查询缓存中的图书详情JSON（使用multiGet实现 pipeline 效果，
减少网络交互）
 List<String> jsons =
redisTemplate.opsForValue().multiGet(keys);

 // 解析JSON为Book对象并收集结果
 List<Book> res = new ArrayList<>();
 for (String j : jsons) {
 // 跳过空值或占位符
 if (j == null || "__NULL__".equals(j)) continue;
 try {
 res.add(objectMapper.readValue(j, Book.class));
 } catch (Exception e) {
 // 解析失败时可跳过或从数据库查询作为降级方案
 }
 }
 return res;
}

/**
 * 增加图书的热度评分
 * @param bookId 图书ID
 * @param delta 增加的评分值（可正可负）

```

```

 */
 @Override
 public void incrementHotScore(Long bookId, double delta) {
 // 操作有序集合，为指定图书ID的分数增加delta值
 redisTemplate.opsForZSet().incrementScore(KEY_HOT,
String.valueOf(bookId), delta);
 }
 }
}

```

### 3) 在 BookServiceImpl 中接入缓存（示例）

修改 `getById` 与 `create/update` 流程：

```

// 注入图书缓存服务，用于处理缓存相关操作
@Resource
private BookCacheService bookCacheService;

/**
 * 根据ID查询图书
 * 采用缓存优先策略，通过缓存旁路 * @param id 图书ID
 * @return 图书对象（可能来自缓存或数据库）
 */
@Override
public Book getById(Long id) {
 // 1. 优先从缓存读取，使用缓存旁路模式
 // 缓存旁路模式逻辑：先查缓存，命中则返回；未命中则查数据库并同步到缓存
 Book b = bookCacheService.getBookWithCacheAside(id);
 return b;
}

/**
 * 创建新图书
 * 先写入数据库，再更新缓存，保证数据一致性
 * @param b 图书对象
 * @return 新创建图书的ID
 */
@Override
public Long create(Book b) {
 // 设置初始可用库存等于总库存（新图书默认全部可借）
 b.setAvailableCopies(b.getTotalCopies());
 // 写入数据库
 bookMapper.insert(b);
 // 写数据库后，直接将新图书信息写入缓存（也可选择删除缓存让后续查询自动同步）
}

```



```
// 此处选择主动写入，避免首次查询缓存未命中的情况，设置缓存有效期为10分钟
bookCacheService.putBookCache(b.getId(), b, 60 * 10);
return b.getId();
}

/**
 * 更新图书信息
 * 先更新数据库，再删除对应缓存，避免缓存与数据库数据不一致
 * @param b 待更新的图书对象
 */
@Override
public void update(Book b) {
 // 更新数据库中的图书信息
 bookMapper.update(b);
 // 更新成功后删除缓存（缓存删除策略）
 // 相比直接更新缓存，删除缓存更简单且不易出错，后续查询会自动从数据库加载最新数据并更新缓存
 bookCacheService.evictBookCache(b.getId());
}
```

## 4) 借书流程中对缓存的处理（BorrowServiceImpl 改动）

在借书成功后，两种做法推荐其一：

- **方案 A（简单且安全）**：DB 更新后 `evictBookCache(bookId)`。下次请求会回填缓存。
- **方案 B（实时性更好）**：DB 更新成功后在 Redis 里原子更新 `cached Book` 的 `availableCopies`，减少回源。需要用 Lua 脚本或读写 JSON 并写会更复杂。

示例采用方案 A（先实现）：

```
// 在 borrowBook 方法最后：
bookMapper.update(book); // 已做
bookCacheService.evictBookCache(bookId);
bookCacheService.incrementHotScore(bookId, 1.0); // 热度 +1
```

如果你想做**原子更新缓存 availableCopies**，可以在 Redis 使用 Lua 脚本读取 JSON、修改字段并写回，或将 `availableCopies` 单独存在 `book:available:{id}` 做原子 DECR/INCR（推荐做法）。

防止缓存问题（穿透 / 击穿 / 雪崩）与实操策略

### 1. 缓存穿透（请求不存在的 key）

- 将空结果写入缓存（`"__NULL__"`）并设置短 TTL（30-60s）。

## 2. 缓存击穿 (热点 key 在 TTL 到期瞬间大量请求)

- 使用分布式互斥锁 (`SET lock:book:{id} NX PX 5000`)，只有拿到锁的线程去 DB 回填，其他线程等待或直接返回旧值/限流。
- 或使用互斥式缓存重建 (single-flight)：先查询缓存，若 miss 则用 SETNX 抢锁并回填。

## 3. 缓存雪崩 (大量 key 同时失效)

- 随机化 TTL (例如  $10\text{min} \pm 0\sim 5\text{min}$ )，避免同一时刻大量过期。

## 4. 缓存穿透/恶意请求

- 对查询参数做白名单/校验，限制频繁无效请求 (IP 限流)。

示例互斥锁 (简化版)：

```
```java
/**
 * 带分布式锁的图书查询方法，用于解决缓存击穿问题（防止高并发下缓存失效时大量请求直击数据库）
 * @param bookId 图书ID
 * @return 图书对象（可能来自缓存或数据库）
 */
public Book getBookWithLock(Long bookId) {
    // 构建图书详情缓存的键
    String key = KEY_BOOK + bookId;
    // 构建分布式锁的键（针对当前图书ID的锁）
    String lockKey = "lock:book:" + bookId;

    // 先从缓存查询，如果命中直接返回
    String cached = redisTemplate.opsForValue().get(key);
    if (cached != null) {
        return parse(cached); // 假设parse方法用于解析JSON为Book对象
        // (类似之前的readValue逻辑)
    }

    // 缓存未命中，尝试获取分布式锁（setIfAbsent：键不存在时才设置，返回true表示获取锁成功）
    // 锁的过期时间设为5秒，防止锁持有者意外崩溃导致死锁
    Boolean ok = redisTemplate.opsForValue().setIfAbsent(lockKey, "1", Duration.ofSeconds(5));

    // 获取锁成功：只有一个线程能进入此分支，负责从数据库加载数据并更新缓存
    if (Boolean.TRUE.equals(ok)) {
        try {
            // 再次检查缓存（双重校验：防止其他线程释放锁后已更新缓存）
            // （注：此处原代码未体现，实际优化中可添加二次检查减少DB压力）
        }
    }
}
```

```

        // 从数据库查询图书
        Book b = bookMapper.findById(bookId);

        // 数据库中不存在该图书：设置空值占位符到缓存，避免重复穿透
        if (b == null) {
            redisTemplate.opsForValue().set(key, "__NULL__",
Duration.ofSeconds(60));
            return null;
        }

        // 数据库查询成功：将图书信息序列化后写入缓存，设置10分钟过期
        redisTemplate.opsForValue().set(key,
objectMapper.writeValueAsString(b), Duration.ofMinutes(10));
        return b;
    } finally {
        // 无论是否发生异常，最终都释放锁（删除锁键）
        redisTemplate.delete(lockKey);
    }
} else {
    // 获取锁失败：说明有其他线程正在加载数据，当前线程等待后重试从缓存获
    取
    try {
        // 等待50毫秒（短时间等待，避免频繁重试）
        Thread.sleep(50);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // 恢复中断状态
    }
    // 重试从缓存获取（此时可能已被其他线程更新）
    return getBookFromCache(bookId);
}
}
}

```

测试 & 监控（本地命令 & CI）

- 本地验证 (redis-cli) :
 - `redis-cli KEYS "book:detail:*"`
 - `redis-cli GET book:detail:1`
 - `redis-cli ZREVRANGE book:hot 0 9 WITHSCORES`
- 查看 info: `redis-cli INFO memory / redis-cli INFO stats`
- 在 CI: 写集成测试 (SpringBootTest 注入 RedisTestContainer 或本地 Redis) 来验证缓存命中逻辑与 evict。

- 监控：建议用 Prometheus + Redis exporter + Grafana。监控 hit/miss 需要在应用层记录（如在缓存命中/未命中时埋点统计）。

运维注意事项（Redis 配置）

- `maxmemory-policy` 视场景选 `volatile-lru` 或 `allkeys-lru`。一般对缓存使用 `volatile-*`，严格区分缓存与持久化数据。
- TTL 与 eviction：确保 Redis 容量足够，否则频繁 evict 会影响命中。
- 生产推荐使用 Redis Cluster 或托管服务（阿里云、AWS ElastiCache），并启用 AUTH。

面试问答速成（常被问到的问题 + 精简答案）

Q：为什么用 Cache-Aside 而不是 Write-Through？

A：Cache-Aside 写入复杂业务更安全（写 DB 首先成功，再失效缓存），避免写失败导致缓存和 DB 不一致。Write-Through 简单但每次写要同步写缓存与 DB，复杂事务易错。

Q：如何保证缓存与 DB 强一致？

A：严格做 DB 写成功后立即删除/更新缓存，或者借助消息队列做最终一致性；但强一致往往牺牲性能，需权衡。

Q：如何防止热点缓存失效时数据库洪峰？

A：采用互斥锁（single-flight）确保只有一个线程重建缓存、并使用随机 TTL 与预热减少同一时刻过期。

Q：为什么用 ZSET 存热度？不是 DB 的 count 字段？

A：ZSET 原子且支持排名查询（ZREVRANGE），在大量并发场景下比不停更新 DB 的计数列更轻量。DB 可做慢周期持久化同步（定时批量落库）。

小结（你面试时可背）

1. 缓存策略用 Cache-Aside：读先查 redis → miss 去 DB → 写回 redis；写操作 DB 成功后 evict cache。
2. 缓存对象：图书详情（短中 TTL）、热门榜（ZSET）、可借副本计数可用 Redis 原子操作。
3. 防护：空值缓存、分布式锁（SET NX）、TTL 随机化、防雪崩、降级。
4. 代码要点：`RedisConfig` + `BookCacheService` 抽象缓存细节，`BookService` 只关心业务，减少耦合。
5. 监控/测试要到位——命中率、miss率、redis memory/eviction。

我即将参加的面试是java岗位的面试，侧重于开发。请告诉我如果面试官说：说一下你在项目中主要做什么内容是什么？简单介绍一下你的项目？在项目中遇到的最大的困难挑战是什么，你是怎么克服的？

太好——我把你要的三问（“你做了什么 / 项目简介 / 最大挑战 & 如何克服”）都整理成**面试友好版**：每题给**一句话电梯答（30s）**、**详细答（1-2min）**、**面试官可能追问点 + 你可补充的技术细节或代码片段**。你可以直接背电梯答开场，随后用详细版扩展；技术细节块在需要时拿出来 show。语言用中文，既专业又不僵硬，适合 Java 开发岗面试。

问 1 — 说一下你在项目中主要做的内容是什么？

电梯答（30s）

“我在图书管理系统里担任项目组长，主要负责后端架构与实现：用 Spring Boot + MyBatis 完成图书增删改查与分页、实现借阅/归还的事务逻辑（并发安全），使用 Redis 做热点缓存与热度排行榜，写单元/集成测试并配合前端/测试同学联调接口与性能测试。”

详细答（1-2min）

“项目我负责整体后端设计与关键模块开发，同时做组内协调和代码评审。具体工作包含：

- 设计并实现用户认证/授权、图书 CRUD、借书/还书/预约等 RESTful 接口（Spring Boot + MyBatis）。
- 借书是关键业务：按副本粒度设计 `book_copies` 表，用事务 + `SELECT ... FOR UPDATE` 在 Service 层锁定副本，保证并发下不会超借；并实现死锁重试策略和单元/集成并发用例验证。
- 在读密集场景用 Redis 做 Cache-Aside（`book:detail:{id}`）和 ZSET 做热门书排行榜，DB 写成功后及时失效缓存以保证最终一致性。
- 自动化：用 JUnit/Mockito 做单元测试，Testcontainers 写集成并发测试；与测试同学用 Postman、JMeter 做接口与压力验证。
- 兼顾工程实践：用 Flyway 管理 schema、用 Swagger 生成接口文档、在 PR 流程中做 Code Review 和接口联调。”

面试官可能追问 & 你该补充

- “你具体负责哪些接口的实现？” → 列举：`POST /api/auth/*`，`GET /api/books`，`POST /api/borrow`，`POST /api/borrow/return`，`GET /api/me/borrow-records` 等。
- “你在组长位置怎么分工？” → 说明你分配任务、安排接口联调、组织周会、合并 PR 与代码规范检查。
- 想要展示代码可以补一句：“比如借书的事务逻辑我在 `BorrowService.borrowBook()` 里实现（带 `@Transactional`），并在

mapper 用 `SELECT ... FOR UPDATE` 锁行。”然后可以出示下面的简短代码片段（看下面）。

问 2 — 简单介绍一下你的项目？（面试官问整体架构/目的）

电梯答（30s）

“这是为校园场景做的图书管理系统，核心目标是支持用户账号、图书检索、并发安全的借还流程和日常运维（缓存、日志、文档、测试），技术栈：Spring Boot、MyBatis、MySQL、Redis、Vue 前端，注重事务安全与性能可测。”

详细答（1-2min）

“整体架构是典型的分层后端 + 单页前端：

- **前端**：Vue.js + Element UI（登录、检索、借阅、我的借阅）
- **后端**：Spring Boot（Controller → Service → Mapper），MyBatis 管理 SQL，Flyway 做 DB 迁移，Redis 做缓存/排行榜；接口文档用 Swagger。
- **数据模型**：`books`（元数据）+ `book_copies`（副本粒度）+ `borrow_records`（借阅日志）+ `users` + `reservations`。
- **关键设计决策**：对借书使用副本级锁定保证一致性；对搜索用 MyBatis 写可调 SQL（短期用 MySQL LIKE 或 FULLTEXT，长期计划接入 ES）；对热点读用 Redis 缓存和 ZSET 做排名。
- **工程化**：写单元/集成测试，和测试组用 JMeter 做并发场景验证，并把常用 Postman 集合留在仓库方便演示。”

面试官可能追问 & 你该补充

- “为什么要做 `book_copies` 而不只用 `available_count`？” → 回答：**副本级能精确管理每一册（扫码、盘点、预约）并能在并发时以行锁方式安全分配**；计数字段更简单但在高并发下需要更多补偿逻辑。
- “搜索用什么策略？” → MVP：MySQL LIKE / FULLTEXT；当搜索成为瓶颈会迁到 Elasticsearch（IK 分词，支持联想和相关度）。
- “怎么部署/运行/升级？” → Flyway 管理迁移，Docker/docker-compose 做 dev 环境，CI（unit + integration + newman）保证质量。

问 3 — 在项目中遇到的最大困难/挑战是什么？你怎么克服的？（用 STAR）

电梯答 (30s)

“最大的挑战是并发借书导致的资源竞争与一致性问题，我通过改变数据模型（副本表）、在 Service 层用事务 + `SELECT ... FOR UPDATE`、实现死锁重试并写并发集成测试验证，最终保证了借书场景的正确性和稳定性。”

详细 STAR 回答 (建议面试时按这套说法)

S (Situation)：项目核心是借书/还书功能，学校里高并发场景会在开学/活动时出现，若控制不好可能出现“同一本副本被多次借出”或库存不一致。

T (Task)：我的任务是保证借书流程在并发下的正确性，同时保持响应性能，不触及过重的分布式架构。

A (Action)：我做了以下具体工作：

1. **数据设计**：把库存从简单计数换成 `book_copies`（每副本一行），便于行级锁定。
2. **事务实现**：在 Service 层用 `@Transactional`，在 MyBatis mapper 写 `SELECT id FROM book_copies WHERE book_id=? AND copy_status=1 LIMIT 1 FOR UPDATE`，在同一事务内更新副本状态并插入 `borrow_records`，最后更新 `books.available_copies`。
3. **异常处理**：捕获死锁异常（MySQL 返回 `ER_LOCK_DEADLOCK`），在调用端做有限次数的指数退避重试。
4. **测试/验证**：用 JUnit + Testcontainers 写并发测试（两个线程同时尝试借同一书），用 JMeter 做压力测试，观察锁等待与成功率，调整索引与 SQL 以减少锁粒度。
5. **性能配套**：为避免读压力过大，我和队友用 Redis 做热点缓存（Cache-Aside），读优先走缓存，写成功后 `evict` 缓存以保证数据最终一致性。

R (Result)：经过实现与多轮测试，借书的并发冲突被理顺（并发请求不会让同一副本被多次借出），接口稳定；我们还能在面试 demo 与老师演示并发用例，项目因此得到高分评价。

面试官可能追问 & 你该补充 (常见追问与简答)

• “为什么不用乐观锁或 Redis 预减库存？”

- 回答要点：我会说“悲观锁（`FOR UPDATE`）在中等并发且数据一致性优先的业务场景最简单可靠；乐观锁适合冲突少的场景，Redis 预减能提供更高吞吐但需要复杂的补偿机制（回滚、消息队列），初期我们优先用悲观锁做稳态实现，后期在压力非常大时可以做演进。”

• “如何处理死锁？”

- 回答要点：短期：捕获异常做退避重试；长期：优化 SQL 与索引，降低锁范围，减少事务持锁时间。

- “如何验证你的解决方案？”

- 回答要点：写了并发集成测试（Testcontainers + 多线程），同时用JMeter做压力测试并观察错误率与延迟，记录并调整（例如把副本索引做成（book_id, copy_status）来加速定位并减少扫描）。

关键代码片段（面试中可以直接念或展示）

Borrow 事务（简短版，可现场念）

```
@Transactional
public Long borrowBook(Long userId, Long bookId) {
    Long copyId =
bookCopyMapper.selectAvailableCopyForUpdate(bookId); // SELECT
... FOR UPDATE
    if (copyId == null) throw new RuntimeException("no available
copy");
    bookCopyMapper.updateCopyStatus(copyId, 2); // 标记借出
    borrowRecordMapper.insert(...);
    bookMapper.updateAvailableCopies(bookId, -1);
    bookCacheService.evictBookCache(bookId); // 写后失效缓存
    return recordId;
}
```

MyBatis mapper (XML) 关键 SQL

```
<select id="selectAvailableCopyForUpdate" resultType="long"
parameterType="long">
    SELECT id FROM book_copies
    WHERE book_id = #{bookId} AND copy_status = 1
    ORDER BY id ASC
    LIMIT 1
    FOR UPDATE
</select>
```

面试展示要点（你该如何组织回答让面试官信服）

1. 开头一句话概括（电梯答），让面试官快速抓住重点。
2. 展开具体工作（职责、技术栈、关键模块），并点名你亲自实现的关键业务（借书事务、缓存、测试）。

3. 用 STAR 讲挑战，过程描述详细（代码/SQL/测试），结果强调“通过测试验证/演示/得分”。
4. 结尾补一句“我如果有机会上线/运维，会继续做 X（比如迁 ES、引入 MQ 做最终一致性、做横向扩展）”，显示工程思考与前瞻性。